UNIVERSIDAD AUTÓNOMA DE SINALOA

FACULTAD DE INFORMÁTICA CULIACÁN

MANUAL DE PRÁCTICAS DE MONGODB

COLECCIÓN, CONSULTA, ACTUALIZACIÓN Y ELIMINACIÓN DE DOCUMENTOS CON MONGODB



ASIGNATURA: Bases de Datos Emergentes

PROGRAMA EDUCATIVO: Licenciatura en Ingeniería en Ciencia de Datos

RESPONSABLE: Dra. Lidia Yadira Pérez Aguilar

FECHA: agosto 2026

DATOS • CONOCIMIENTO • DECISIONES

Tecnología que transforma información en valor

PRÁCTICA 7. Aggregation Pipeline II

Objetivo

Aplicar etapas avanzadas del framework de agregación de MongoDB para transformar campos, descomponer arrays y analizar información temporal mediante $project, $unwind, $year, $month, $multiply, $group, $sort y $limit.

Actividades

- Ejecutar el código en el orden indicado y verificar los resultados.
- Registrar evidencia de las consultas ejecutadas.
- Guardar el código en un documento .js
- Subirlo al repositorio GitHub

Entregable

- Código completo y ejecutable..
- PDF con las respuestas a los ejercicios..

0. Preparar el entorno

- **Inicie MongoDB y posteriormente mongosh.**
- **Seleccione la base de datos utilizada en las prácticas anteriores**

```
use ventas_online
```

- **Compruebe las colecciones existentes:**

```
show collections
```

Verificar:
db.productos.countDocuments()
db.clientes.countDocuments()
db.proveedores.countDocuments()
db.ventas.countDocuments()

1. Operador $project

Permite procesar documentos mediante una secuencia de etapas denominada pipeline. Hasta ahora hemos utilizado principalmente `find()` para recuperar documentos.

El método:

$project permite seleccionar, ocultar o crear campos dentro de un pipeline.

```
db.ventas.aggregate([
    {
        $project: {
            _id: 0,
            venta_id: 1,
            cliente_id: 1,
            fecha: 1,
            total: 1
        }
    }
])
```

2. Crear un nuevo campo calculado

$project también permite crear nuevos campos a partir de operaciones o expresiones realizadas sobre campos existentes.

Agregar un campo denominado iva_estimado, equivalente al 16 % del valor de total:

```
db.ventas.aggregate([
    {
        $project: {
            _id: 0,
            venta_id: 1,
            total: 1,
            iva_estimado: {
                $multiply: ["$total", 0.16]
            }
        }
    }
])
```

3. Trabajar con fechas

3.1 Obtener el año de una venta

```
db.ventas.aggregate([
    {
        $project: {
            _id: 0,
            venta_id: 1,
            fecha: 1,
            anio: {
                $year: "$fecha"
            }
        }
    }
])
```

3.2 Obtener el mes

```
db.ventas.aggregate([
```

```
    {
        $project: {
            _id: 0,
            venta_id: 1,
            fecha: 1,
            mes: {
                $month: "$fecha"
            }
        }
    }
])
```

3.3 Obtener año y mes

```
db.ventas.aggregate([
    {
        $project: {
            _id: 0,
            venta_id: 1,
            anio: {
                $year: "$fecha"
            },
            mes: {
                $month: "$fecha"
            },
            total: 1
        }
    }
])
```

4. Agrupar ventas por mes

Crear un pipeline de tres etapas:

$project



$group



$sort

```
db.ventas.aggregate([
    {
        $project: {
            anio: {
                $year: "$fecha"
            },
            mes: {
                $month: "$fecha"
            },
            total: 1
        }
    },
    {
        $group: {
            _id: {
```

```
                anio: "$anio",
                mes: "$mes"
            },
            ventas: {
                $sum: 1
            },
            ingreso_total: {
                $sum: "$total"
            }
        }
    },
    {
        $sort: {
            "_id.anio": 1,
            "_id.mes": 1
        }
    }
])
```

5. Introducción a $unwind

$unwind permite descomponer un campo que contiene un array y generar un documento de salida por cada elemento del array.

Por ejemplo, una venta puede contener varios productos dentro del array items:

```
items: [
    {
        sku: "P001",
        cantidad: 1,
        precio: 1200
    },
    {
        sku: "P002",
        cantidad: 2,
        precio: 350
    }
]
```

6. Descomponer el array items

Utilizar $unwind para descomponer el array items:

```
db.ventas.aggregate([
    {
        $unwind: "$items"
    }
])
```

`$unwind` genera un documento de salida por cada elemento contenido en el array.

Por ejemplo, si originalmente tenemos una venta:

```
{
    venta_id: "V001",
    items: [
        {sku: "P001", cantidad: 1, precio: 1200},
        {sku: "P002", cantidad: 2, precio: 350}
    ]
}
```

Después de aplicar:

```
{$unwind: "$items"}
```

conceptualmente obtenemos:

```
{
    venta_id: "V001",
    items: {
        sku: "P001",
        cantidad: 1,
        precio: 1200
    }
}

{
    venta_id: "V001",
    items: {
        sku: "P002",
        cantidad: 2,
        precio: 350
    }
}
```

7. Mostrar los productos contenidos en las ventas

```
db.ventas.aggregate([
    {
        $unwind: "$items"
    },
    {
        $project: {
            _id: 0,
            venta_id: 1,
            cliente_id: 1,
            sku: "$items.sku",
            cantidad: "$items.cantidad",
            precio: "$items.precio"
        }
    }
])
```

8. Calcular unidades vendidas por producto

```
db.ventas.aggregate([
    {
        $unwind: "$items"
    },
```

```
    {
        $group: {
            _id: "$items.sku",
            unidades_vendidas: {
                $sum: "$items.cantidad"
            }
        }
    },
    {
        $sort: {
            unidades_vendidas: -1
        }
    }
])
```

9. Operador $multiply.

Calcular ingreso por producto. Cada producto puede aportar: cantidad × precio

```
db.ventas.aggregate([
    {
        $unwind: "$items"
    },
    {
        $group: {
            _id: "$items.sku",
            unidades_vendidas: {
                $sum: "$items.cantidad"
            },
            ingreso_producto: {
                $sum: {
                    $multiply: [
                        "$items.cantidad",
                        "$items.precio"
                    ]
                }
            }
        }
    },
    {
        $sort: {
            ingreso_producto: -1
        }
    }
])
```

10. Obtener los productos con mayor ingreso

Obtener los 5 productos que generaron mayor ingreso.

```
db.ventas.aggregate([
    {
        $unwind: "$items"
    },
    {
```

```
        $group: {
            _id: "$items.sku",
            unidades_vendidas: {
                $sum: "$items.cantidad"
            },
            ingreso_producto: {
                $sum: {
                    $multiply: [
                        "$items.cantidad",
                        "$items.precio"
                    ]
                }
            }
        }
    },
    {
        $sort: {
            ingreso_producto: -1
        }
    },
    {
        $limit: 5
    }
])
```

11. Analizar únicamente ventas completadas

Obtener la siguiente secuencia:

$match

↓

$unwind

↓

$group

↓

$sort

```
db.ventas.aggregate([
    {
        $match: {
            estado: "completada"
        }
    },
    {
        $unwind: "$items"
    },
    {
        $group: {
            _id: "$items.sku",
            unidades: {
                $sum: "$items.cantidad"
            }
        }
    },
    {
        $sort: {
            unidades: -1
        }
    }
```

```
])
```

12. Combinar análisis temporal y ventas

Calcular ingresos mensuales solamente para ventas completadas:

```
db.ventas.aggregate([
    {
        $match: {
            estado: "completada"
        }
    },
    {
        $project: {
            anio: {
                $year: "$fecha"
            },
            mes: {
                $month: "$fecha"
            },
            total: 1
        }
    },
    {
        $group: {
            _id: {
                anio: "$anio",
                mes: "$mes"
            },
            ventas: {
                $sum: 1
            },
            ingreso: {
                $sum: "$total"
            }
        }
    },
    {
        $sort: {
            "_id.anio": 1,
            "_id.mes": 1
        }
    }
])
```

13. Calcular la cantidad total de unidades vendidas

```
db.ventas.aggregate([
    {
        $unwind: "$items"
    },
    {
        $group: {
            _id: null,
            unidades_totales: {
                $sum: "$items.cantidad"
```

```
            }
        }
    }
])
```

13. Calcular el promedio de unidades vendidas por producto

```
db.ventas.aggregate([
    {
        $unwind: "$items"
    },
    {
        $group: {
            _id: "$items.sku",
            unidades: {
                $sum: "$items.cantidad"
            }
        }
    },
    {
        $group: {
            _id: null,
            promedio_unidades_por_producto: {
                $avg: "$unidades"
            }
        }
    }
])
```

Ejercicios de la Práctica 7

1. Mostrar venta_id, cliente_id, fecha y total mediante $project.
2. Crear un campo denominado iva_estimado equivalente al 16 % del total de cada venta.
3. Mostrar el año correspondiente a cada venta.
4. Mostrar el mes correspondiente a cada venta.
5. Mostrar año, mes y total de cada venta.
6. Agrupar las ventas por año y mes.
7. Calcular cuántas ventas existen por mes.
8. Calcular el ingreso total por mes.
9. Descomponer el array items utilizando $unwind.
10. Mostrar venta_id, sku, cantidad y precio de cada producto vendido.
11. Calcular las unidades vendidas por sku.
12. Ordenar los productos de mayor a menor según unidades vendidas.
13. Calcular el ingreso generado por cada producto.
14. Obtener los 5 productos con mayor ingreso.
15. Obtener los 10 productos con mayor cantidad de unidades vendidas.
16. Analizar únicamente las ventas completadas y calcular las unidades vendidas por producto.
17. Calcular los ingresos mensuales únicamente de las ventas completadas.
18. Calcular el total de unidades vendidas en todas las operaciones.
19. Calcular el promedio de unidades vendidas por producto.
20. Diseñar un pipeline propio que incluya al menos $match, $unwind, $group y $sort, y explicar qué indicador comercial obtiene.

- Sube a tu repositorio:

    - El archivo: Practica07-Aggregation-Pipeline-II.js
    - El archivo: Practica07-Aggregation-Pipeline-II.pdf (Con la portada)